

## EXP NO 11:

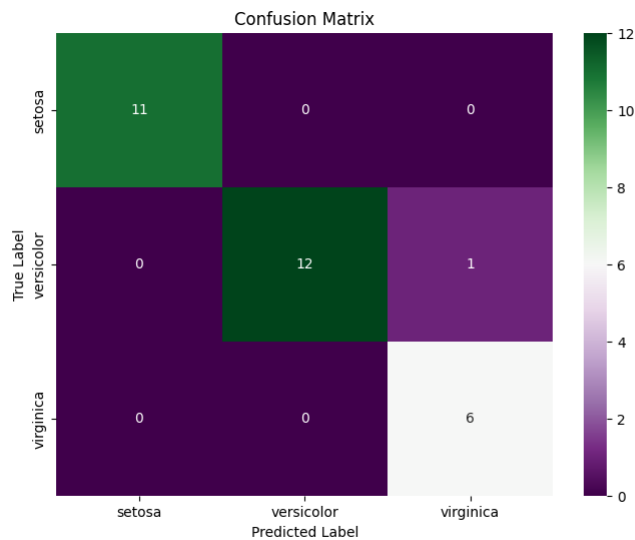
[18]  
✓ Os

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

iris = load_iris()

data = pd.DataFrame(data=iris.data, columns=iris.feature_names)
data['species'] = iris.target
X = data.drop('species', axis=1)
y = data['species']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=1)
model = DecisionTreeClassifier()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("accuracy:", accuracy)
conf_matrix = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='PRGn',
            xticklabels=iris.target_names, yticklabels=iris.target_names)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix')
plt.show()
```

... accuracy: 0.9666666666666667

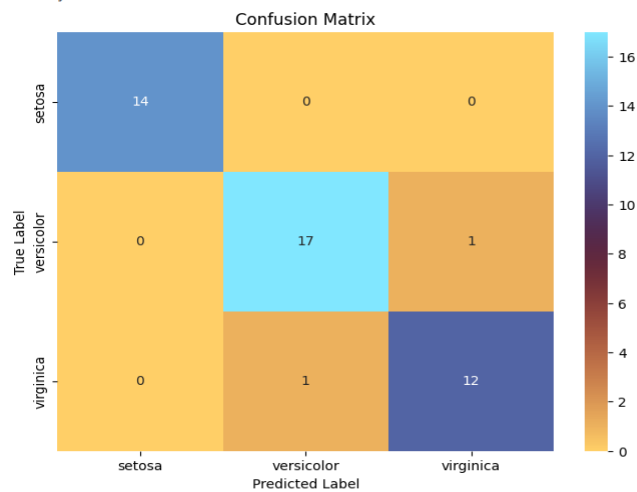


## EXP NO 12

```
[19]  import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
iris=load_iris()
data=pd.DataFrame(data=iris.data,columns=iris.feature_names)
data['Species']=iris.target
x=data.drop('Species',axis=1)
y=data['Species']
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.3,random_state=1)
model=RandomForestClassifier(n_estimators=100,random_state=1)
model.fit(x_train,y_train)
y_pred=model.predict(x_test)
accuracy=accuracy_score(y_test,y_pred)
print("accuracy:",accuracy)
conf_matrix = confusion_matrix(y_test,y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='magma',
            xticklabels=iris.target_names, yticklabels=iris.target_names)
plt.xlabel('Predicted Label')

plt.ylabel('True Label')
plt.title('Confusion Matrix')
plt.show()
```

... accuracy: 0.9555555555555556



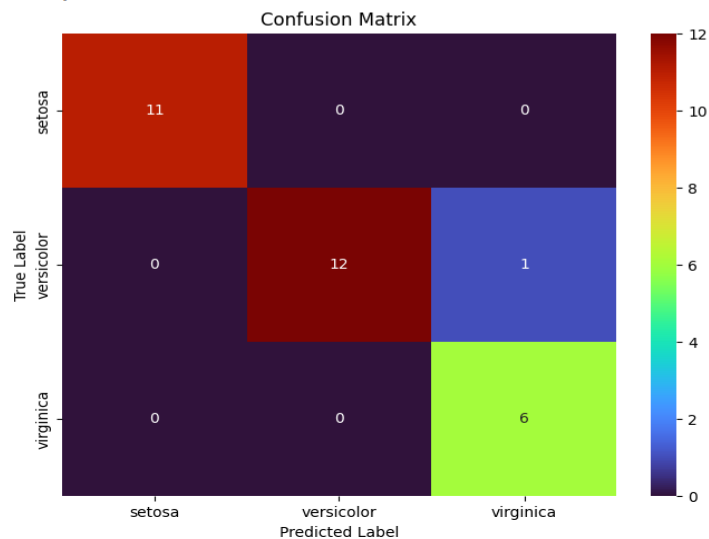
## EXP NO 13

[28]  
✓ Os

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix
iris=load_iris()
data=pd.DataFrame(data=iris.data,columns=iris.feature_names)
data['Species']=iris.target
x=data.drop('Species',axis=1)
y=data['Species']
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=1)
model=SVC(kernel='poly',random_state=1)
model.fit(x_train,y_train)
y_pred=model.predict(x_test)
accuracy=accuracy_score(y_test,y_pred)
print("accuracy:",accuracy)
conf_matrix = confusion_matrix(y_test,y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='turbo',
            xticklabels=iris.target_names, yticklabels=iris.target_names)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix')
plt.show()
```

... accuracy: 0.9666666666666667



## EXP NO 14

```
[23]
✓ Os
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
def mean_squared_error(y_true, y_predicted):
    cost = np.sum((y_true - y_predicted)**2) / len(y_true)
    return cost
def gradient_descent(x, y, iterations=1000, learning_rate=0.01,
                    stopping_threshold=1e-6):
    current_weight = 0.0
    current_bias = 0.0

    n = float(len(x))
    costs = []
    previous_cost = None
    for i in range(iterations):
        y_predicted = current_weight * x + current_bias
        current_cost = mean_squared_error(y, y_predicted)
        if previous_cost and abs(previous_cost - current_cost) <= \
            stopping_threshold:
            break
        previous_cost = current_cost
        costs.append(current_cost)
        weight_derivative = -(2/n) * sum(x * (y - y_predicted))
        bias_derivative = -(2/n) * sum(y - y_predicted)
        current_weight = current_weight - learning_rate * weight_derivative
        current_bias = current_bias - learning_rate * bias_derivative
        if i % 100 == 0:
            print(f"Iteration {i+1}: Cost {current_cost}, Weight {current_weight}, Bias {current_bias}")
    plt.figure(figsize=(8,6))
    plt.plot(range(len(costs)), costs, 'r.')
    plt.title("Cost vs Iterations")
    plt.xlabel("Iterations")
    plt.ylabel("Cost")
    plt.show()
    return current_weight, current_bias
def main():

    X = np.array([32.5, 53.4, 61.5, 47.4, 59.8, 55.1, 52.2, 39.2, 48.1, 52.5, 45.4,
                  54.3, 44.1, 58.1, 56.7, 48.9, 44.6, 60.2, 45.6, 38.8])
    Y = np.array([31.7, 68.7, 62.5, 71.5, 87.2, 78.2, 79.6, 59.1, 75.3, 71.3, 55.1,
                  82.4, 62.0, 75.3, 81.4, 60.7, 82.8, 97.3, 48.8, 56.8])
    scaler = StandardScaler()
    X_normalized = scaler.fit_transform(X.reshape(-1, 1)).flatten()
    estimated_weight, estimated_bias = gradient_descent(X_normalized, Y,
                                                    iterations=2000, learning_rate=0.01)
    print(f"Estimated Weight: {estimated_weight}, Estimated Bias: {estimated_bias}")
    Y_pred = estimated_weight * X_normalized + estimated_bias
    plt.figure(figsize=(8,6))
    plt.scatter(X, Y, color='black', label='Data Points', marker='*')
    plt.plot(X, Y_pred, color='red', linestyle='--', label='Fitted Line')
    plt.xlabel("X")
    plt.ylabel("Y")
    plt.title("Linear Regression using Gradient Descent")
    plt.legend()
    plt.show()

if __name__ == "__main__":
    main()
```

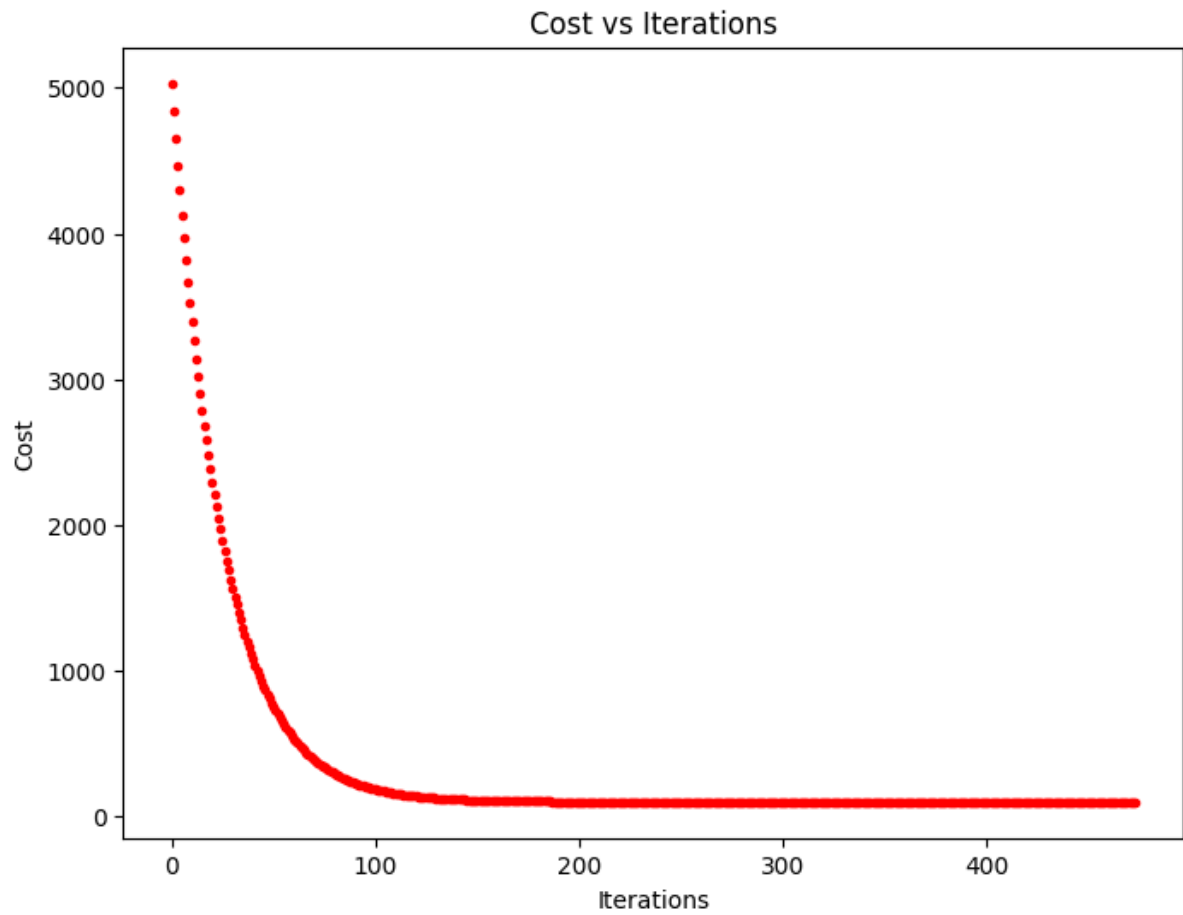
Iteration 1: Cost 5031.3015, Weight 0.21744528996901658, Bias 1.3877

Iteration 101: Cost 185.56941123866557, Weight 9.459227106883091, Bias 60.3672282719577

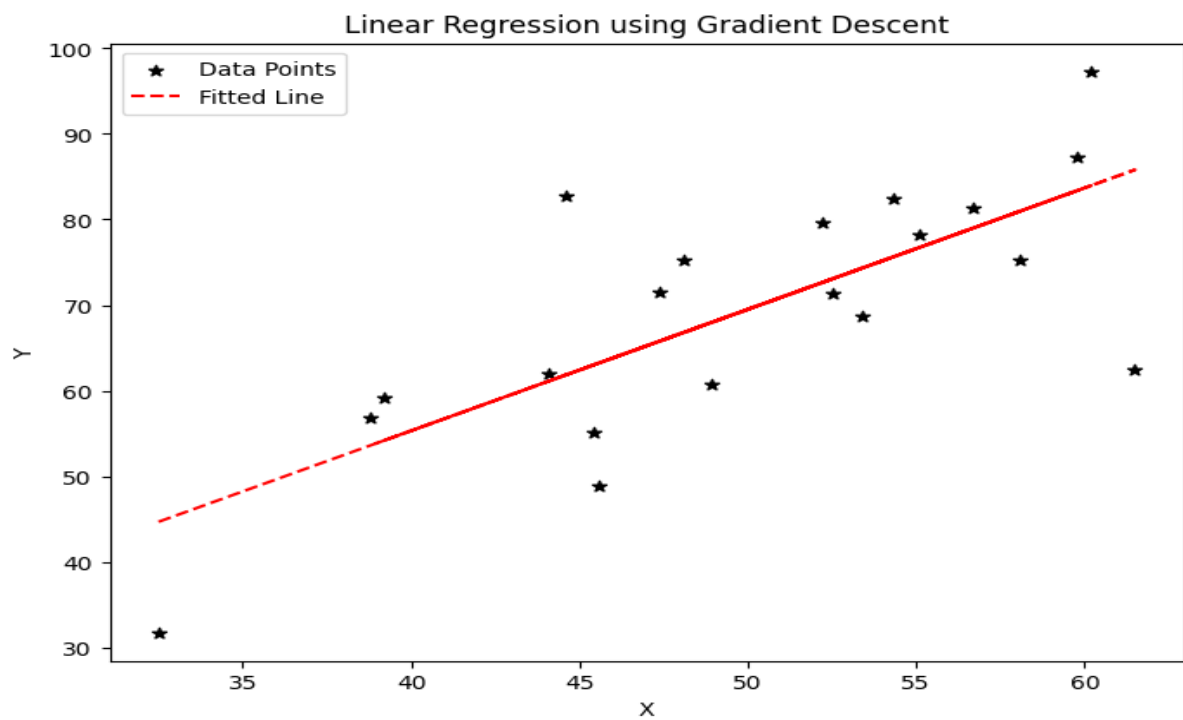
Iteration 201: Cost 100.34293399589959, Weight 10.684868107118445, Bias 68.18906711826675

Iteration 301: Cost 98.84397526476002, Weight 10.847412072256065, Bias 69.22639591234459

Iteration 401: Cost 98.81761165863254, Weight 10.868968580725983, Bias 69.36396599633204



Estimated Weight: 10.87151030998278, Estimated Bias: 69.38018689350649



## EXP NO 15

```
27] import cv2
23s import numpy as np
from matplotlib import pyplot as plt
from google.colab import drive
# Mount Google Drive
drive.mount('/content/drive')
# For Colab: !wget https://example.com/cataract_image.jpg -O image.jpg
#(replace with actual URL)
img = cv2.imread('/content/drive/MyDrive/dog.jpeg') # Replace with your path
# Check if the image was loaded successfully
if img is None:
    print("Error: Could not load image. Please check the file path.")
else:
    rgb_img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    pixels = np.float32(rgb_img.reshape((-1, 3)))
    criteria = (cv2.TERM_CRITERIA_EPS +
                cv2.TERM_CRITERIA_MAX_ITER, 100, 0.2)

    K = 3
    _, labels, centers = cv2.kmeans(pixels, K, None, criteria, 10,
                                   cv2.KMEANS_RANDOM_CENTERS)

    centers = np.uint8(centers)
    segmented_img = centers[labels.flatten()].reshape(rgb_img.shape)
    plt.figure(figsize=(10, 5))
    plt.subplot(121)
    plt.imshow(rgb_img)
    plt.title('Original Image')
    plt.axis('off')

    plt.subplot(122)
    plt.imshow(segmented_img)
    plt.title('Segmented Image (K-means)')
    plt.axis('off')
    plt.tight_layout()
    plt.show()
```